

Reviewer Pack — Minimal Number of Integral Points in Affine Space and Resolu...

Entry 58dc506e170f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 13:54:01 UTC

Minimal Number of Integral Points in Affine Space and Resolution Proof Width

Entry ID: 58dc506e170f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 13:54:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Integral Points in Affine Space) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean formula φ with n variables, the resolution proof width $w(\varphi)$ is upper-bounded by the number of integral points within an n -dimensional affine space containing φ 's feasible region, i.e., $w(\varphi) = O(|Z^n \cap [0,1]^n|)$, where Z is the set of integers and $|A|$ denotes the cardinality of a set A .

Rationale (proposer's reasoning):

The study of integral points in affine spaces has connections to Diophantine geometry, which could potentially provide a combinatorial interpretation for proof complexity. If the number of integral points is small, it may indicate that the resolution proof can be short, suggesting a link between geometric properties and proof complexity.

Taxonomy category: AffineGeometry_ResolutionProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 55d24b07a0f30126

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if, for all $n \geq 2$, the mean resolution proof width $w(\varphi)$ across a sample of random Boolean formulas φ with n variables is less than or equal to three times the mean number of integral points in the feasible region $|Z^n \cap [0,1]^n|$, calculated across at least 100 independent seeds. The conjecture is falsified if any seed produces a resolution proof width greater than ten times the corresponding mean number of integral points.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - integral points AND resolution proof width AND affine space - Boolean formula AND resolution proof width AND integral points in affine space - resolution proof complexity AND upper bound AND number of integral points in affine space

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        return ''.join(random.choice('01') for _ in range(2**n))

    def count_integral_points(n):
        return sum(1 for i in range(2**n) if all(int(bit) >= 0 and int(bit) <= 1 for bit in format(i, '0{}b'.format(2**n))))

    def resolution_proof_width(formula):
        # Simplified DPLL solver to estimate proof width
        clauses = formula.split()
        stack = []
        literals = set()

        def dpll():
            if not clauses:
                return len(stack)
            literal = next((lit for lit in literals if all(lit not in clause and -lit not in clause for clause in clauses)))
            if literal is None:
                return float('inf')
            stack.append(literal)
            literals.remove(literal)
            new_clauses = []
            for clause in clauses:
                if literal in clause:
                    continue
                new_clause = clause.replace(literal, '')
                new_clauses.append(new_clause)
            clauses = new_clauses
            return dpll()

        return dpll()

    return {'formula': generate_boolean_formula(20), 'points': count_integral_points(20), 'width': resolution_proof_width(generate_boolean_formula(20))}
```

```

        elif -literal in clause:
            new_clauses.extend(clause.split()[::-1])
        else:
            new_clauses.append(clause.replace(str(-literal), ''))
    width = dpll()
    stack.pop()
    literals.add(literal)
    return width

    return dpll()

n_values = [5, 10, 15, 20, 30, 40]
total_width = 0
total_points = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Sample 5 instances per size
        formula = generate_boolean_formula(n)
        points = count_integral_points(n)
        width = resolution_proof_width(formula)
        if width == float('inf'):
            continue
        total_width += width
        total_points += points
        instances_tested += 1

mean_width = total_width / instances_tested
mean_points = total_points / instances_tested
conjecture_holds = mean_width <= 3 * mean_points
counterexample = "" if conjecture_holds else f"mean_width={mean_width}, mean_points={mean_points}"

return {
    "metric_name": "resolution_proof_width",
    "metric_value": mean_width,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(2, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(res["metric_value"] for res in results) / len(results)
    std_dev = math.sqrt(sum((res["metric_value"] - mean_width)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_dev} support_fraction={support_fraction}")

```

```

elif any(res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_width} std={std_dev} support_fraction={support_fraction}")
else:
    first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15277
2	preregistration	ollama_remote	glm4:latest	0	0	10273
3	novelty	ollama_remote	glm4:latest	0	0	8319
4	novelty	ollama_remote	glm4:latest	0	0	9088
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14673
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9621
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39062
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12040
9	judge	ollama_remote	glm4:latest	0	0	22249

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140602 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/58dc506e170f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/58dc506e170f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/58dc506e170f.tar.gz` (if generated)